In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
```

In [2]:
```
from google.colab import files

uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving sales_data_sample.csv to sales_data_sample.csv
```

In [3]:
```
filename = next(iter(uploaded))

df = pd.read_csv(filename, encoding="latin1")

print("Dataset shape:", df.shape)
display(df.head())
```

```
Dataset shape: (2823, 25)
```

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDI
0	10107	30	95.70	2	2871.00	2/
1	10121	34	81.35	5	2765.90	5
2	10134	41	94.74	2	3884.34	7
3	10145	45	83.26	6	3746.70	8/
4	10159	49	100.00	14	5205.27	10/

5 rows × 25 columns

In [4]:
```
features = [
    "QUANTITYORDERED",
    "PRICEEACH",
    "SALES",
    "MSRP",
    "MONTH_ID",
    "YEAR_ID"
]

X = df[features].copy()
```

```
# Convert features to numeric and handle missing values
X = X.apply(pd.to_numeric, errors="coerce")
X = X.replace([np.inf, -np.inf], np.nan)
X = X.fillna(X.median())

print("Features selected:", features)
print("Missing values:", X.isnull().sum().sum())
display(X.head())
```

```
Features selected: ['QUANTITYORDERED', 'PRICEEACH', 'SALES', 'MSRP', 'MONTH_ID',
'YEAR_ID']
Missing values: 0
```

	QUANTITYORDERED	PRICEEACH	SALES	MSRP	MONTH_ID	YEAR_ID
0	30	95.70	2871.00	95	2	2003
1	34	81.35	2765.90	95	5	2003
2	41	94.74	3884.34	95	7	2003
3	45	83.26	3746.70	95	8	2003
4	49	100.00	5205.27	95	10	2003

In [5]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

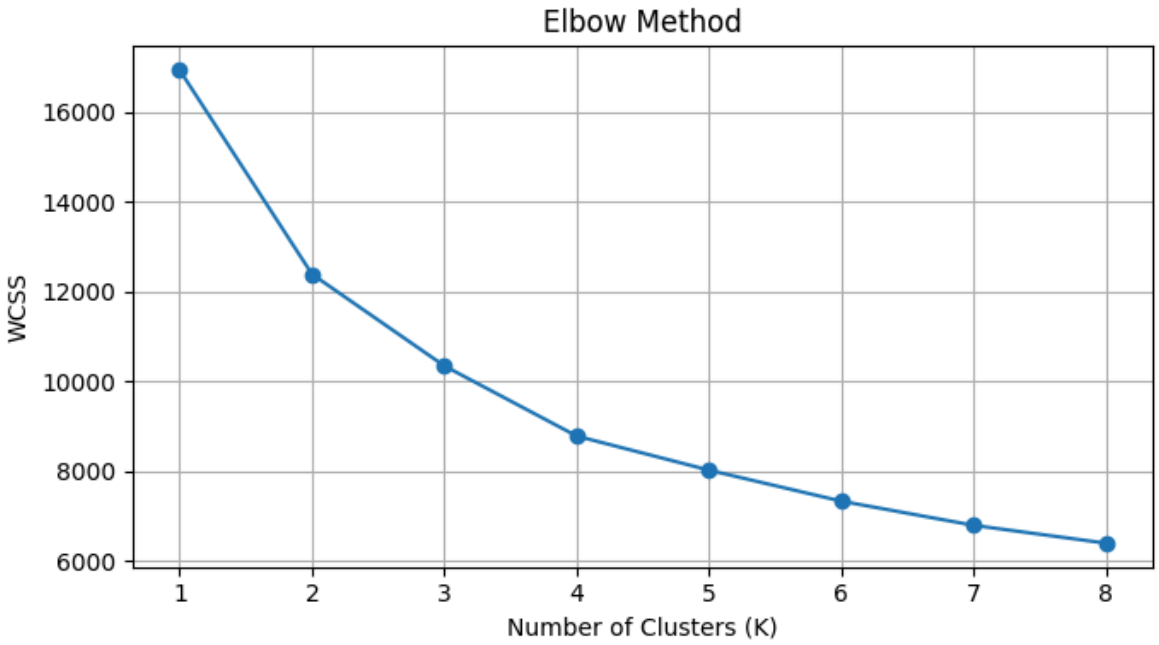
print("Normalized data shape:", X_scaled.shape)
```

```
Normalized data shape: (2823, 6)
```

In [6]:
```
wcss = []

for k in range(1, 9):
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
    model.fit(X_scaled)
    wcss.append(model.inertia_)

plt.figure(figsize=(7, 4))
plt.plot(range(1, 9), wcss, marker="o")
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.xticks(range(1, 9))
plt.grid(True)
plt.tight_layout()
plt.savefig("elbow_method.png", dpi=200)
plt.show()
```



In [7]:
```
k = 3

kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

df["Cluster"] = kmeans.fit_predict(X_scaled)

print("K-Means clustering completed.")
print("\nNumber of records in each cluster:")
print(df["Cluster"].value_counts().sort_index())
```

```
K-Means clustering completed.

Number of records in each cluster:
Cluster
0     658
1    1194
2     971
Name: count, dtype: int64
```

In [8]:
```
wcss_final = kmeans.inertia_
silhouette = silhouette_score(X_scaled, df["Cluster"])

print("Number of clusters:", k)
print("WCSS:", round(wcss_final, 2))
print("Silhouette Score:", round(silhouette, 4))
```

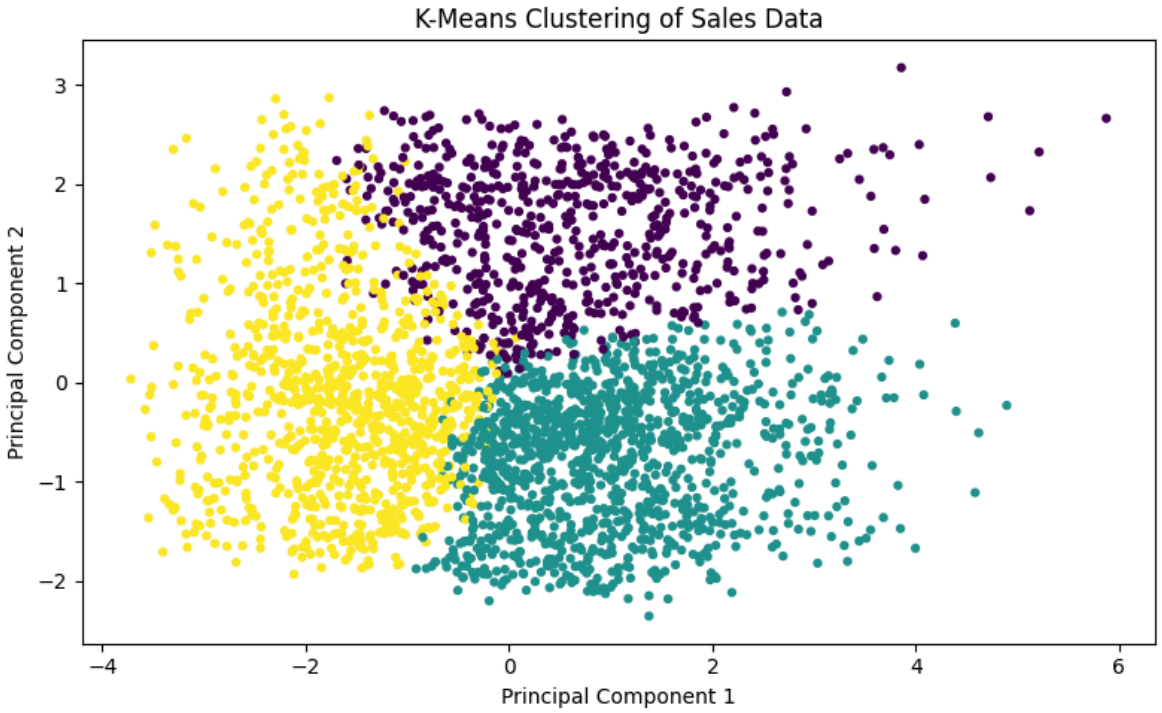
```
Number of clusters: 3
WCSS: 10339.02
Silhouette Score: 0.238
```

In [9]:
```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 5))
plt.scatter(
    X_pca[:, 0],
```

```
    X_pca[:, 1],
    c=df["Cluster"],
    cmap="viridis",
    s=12
)

plt.title("K-Means Clustering of Sales Data")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.tight_layout()
plt.savefig("sales_clusters.png", dpi=200)
plt.show()
```



In [10]:
```
summary = df.groupby("Cluster")[
    ["SALES", "QUANTITYORDERED", "PRICEEACH"]
].mean().round(2)

print("Cluster-wise Sales Analysis:")
display(summary)
```

Cluster-wise Sales Analysis:

Cluster	SALES	QUANTITYORDERED	PRICEEACH
0	4155.49	37.23	91.77
1	4475.08	35.28	97.60
2	2013.46	33.41	61.03

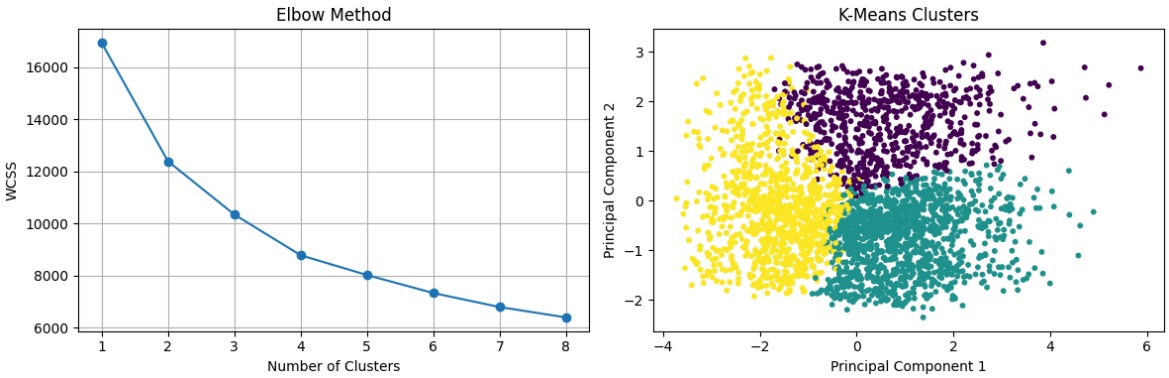
In [11]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(range(1, 9), wcss, marker="o")
axes[0].set_title("Elbow Method")
axes[0].set_xlabel("Number of Clusters")
axes[0].set_ylabel("WCSS")
```

```
axes[0].grid(True)

axes[1].scatter(
    X_pca[:, 0], X_pca[:, 1],
    c=df["Cluster"], cmap="viridis", s=10
)
axes[1].set_title("K-Means Clusters")
axes[1].set_xlabel("Principal Component 1")
axes[1].set_ylabel("Principal Component 2")

plt.tight_layout()
plt.savefig("sales_clustering_result.png", dpi=300, bbox_inches="tight")
plt.show()
```



In [ ]: